Unit 4

Memory Management

Course Outcome 4

Calculate the efficiency of different memory management techniques

Unit No. 4

- **Memory Management:** Main Memory management requirements, memory partitioning techniques static and dynamic, Free space management Techniques — Bitmap, linked list.
- Paging: concept, structure of page table, swapping, Segmentation.
- Virtual Memory: Background, Demand paging, Page Replacement Algorithms: FIFO, LRU, Optimal.

4.1 Main Memory in Computer System



Stores Data and Instructions

1 What is Main Memory ?

Main Memory is the **primary storage area** in a computer system where data and instructions are stored **temporarily** while they are being processed by the CPU.

2 Role of Main Memory

- Stores data and instructions currently in use.
- Provides **fast access** to data for the CPU.
- Supplies data and instructions to the CPU when required.
- Works faster than secondary storage devices.

3 Characteristics

- **Fast** access speed
- **Volatile** memory (data is lost when power is off)
- **Smaller** storage capacity compared to secondary memory
- Stores data **temporarily**

4 Types of Main Memory

Main Memory

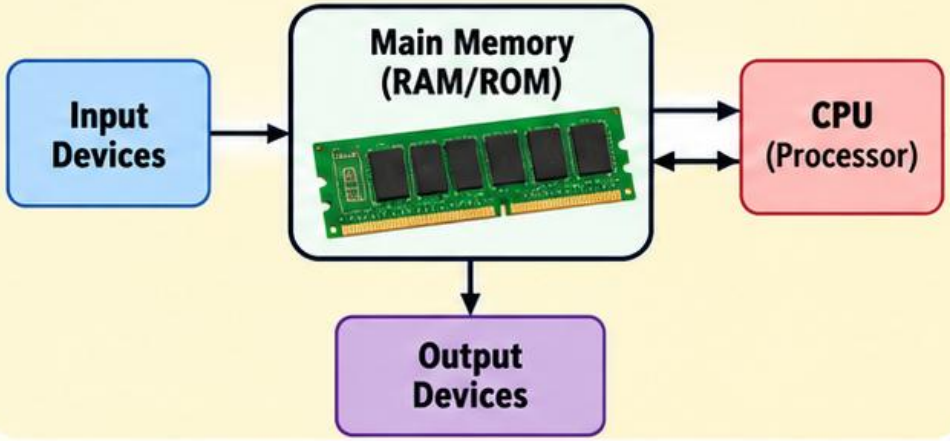
RAM (Random Access Memory)

- Read and write operation
- Volatile memory
- Used to store data and instructions currently in use

ROM (Read Only Memory)

- Read operation only
- Non-volatile memory
- Stores permanent instructions (e.g. BIOS)

5 Main Memory in Computer System

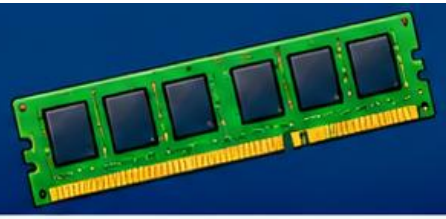


6 Example



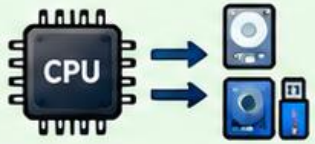
RAM Module ROM Chip

4.1 Memories



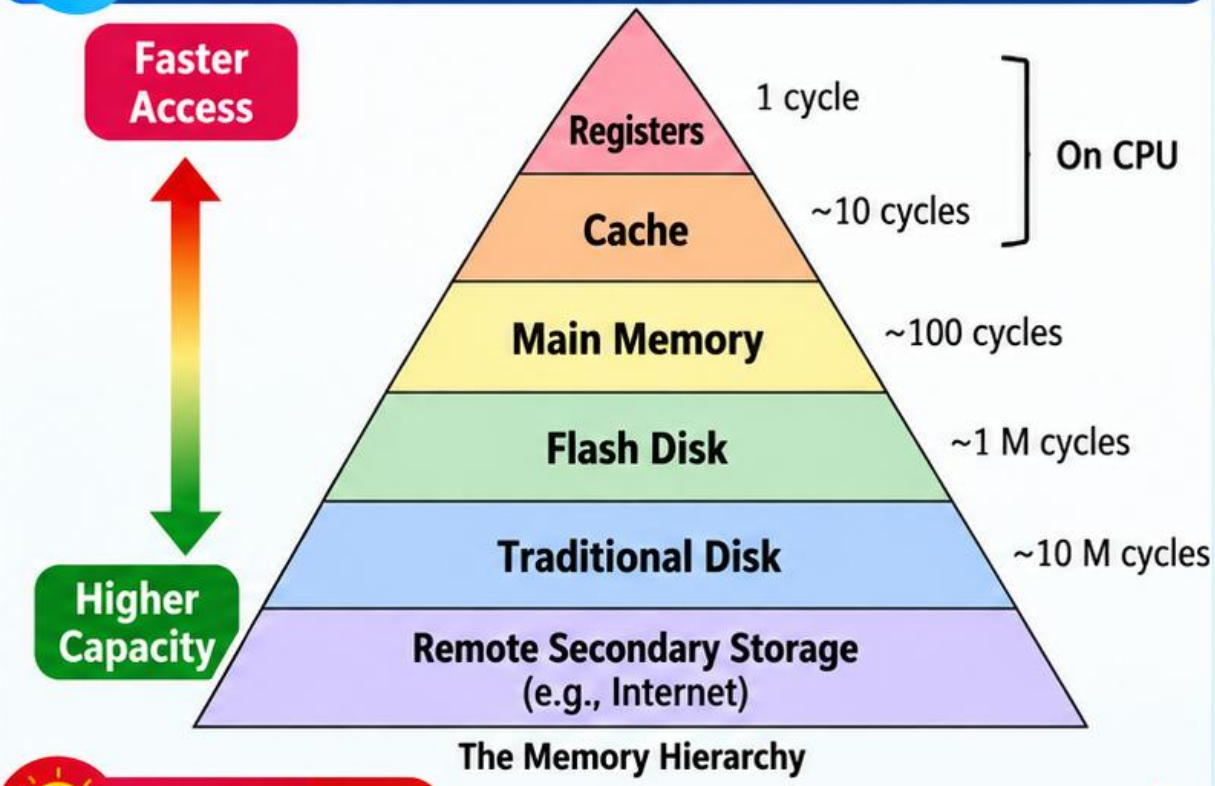


1 What is Main Memory?

1.  The main memory is **central** to the operation of a **Modern Computer.**
2.  Main Memory is a **large array of words or bytes**, ranging in size from hundreds of thousands to billions.
3. 

Main memory is a repository of **rapidly available information** shared by the **CPU** and **I/O devices.**
4.  Main memory is the place where **programs and information are kept** when the processor is effectively utilizing them.
5.  Main memory is associated with the processor, so moving instructions and information into and out of the processor is **extremely fast.**
6. RAM Main memory is also known as **RAM (Random Access Memory).**
7.  This memory is **volatile**. RAM loses its data when a power interruption occurs.

2 Memory Hierarchy



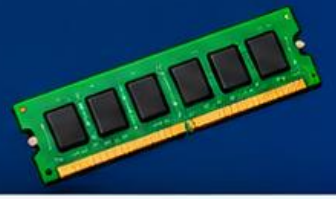
The Memory Hierarchy

Remember:



Main Memory = RAM = Fast + Volatile

4.1.1 Memory management

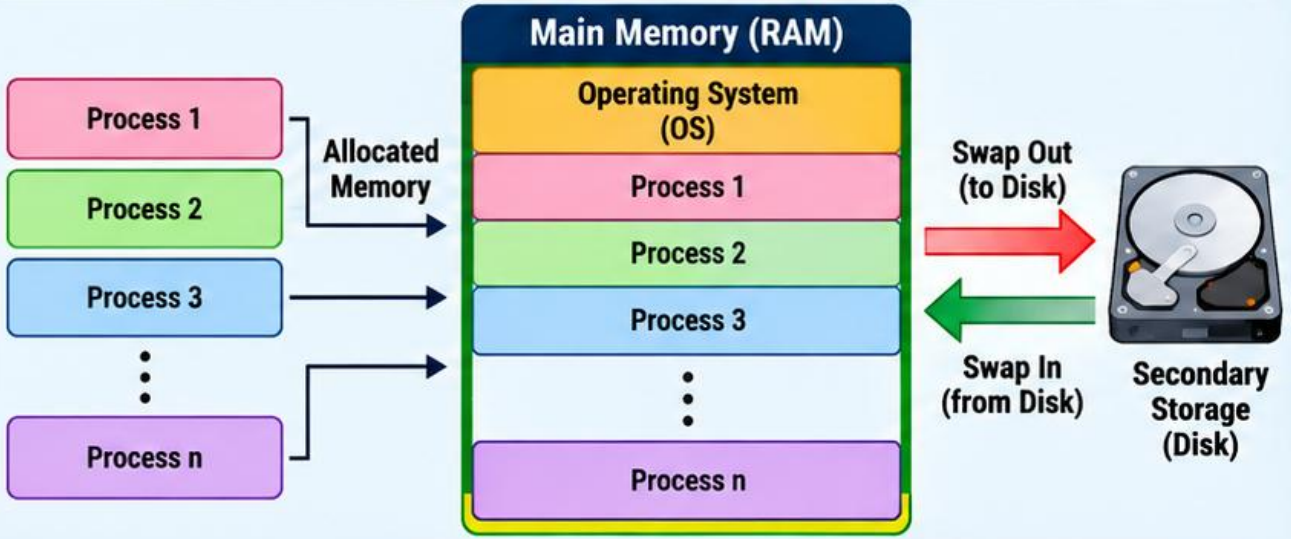




1 What is Memory Management?

- Memory management is the process of **controlling and coordinating** a computer's main memory.
- It ensures that blocks of memory space are properly managed and allocated so the **operating system (OS)**, **applications** and other running **processes** have the memory they need to carry out their operations.

Example: Memory Management in a Multiprogramming System



2 Memory Management: Requirements

1. **Support Multiple Programs**
   Allows multiple programs to be loaded and executed at the same time in memory.

2. **Proper Allocation and Management**
   Ensures that memory space is properly managed and allocated to the OS, applications and other running processes.

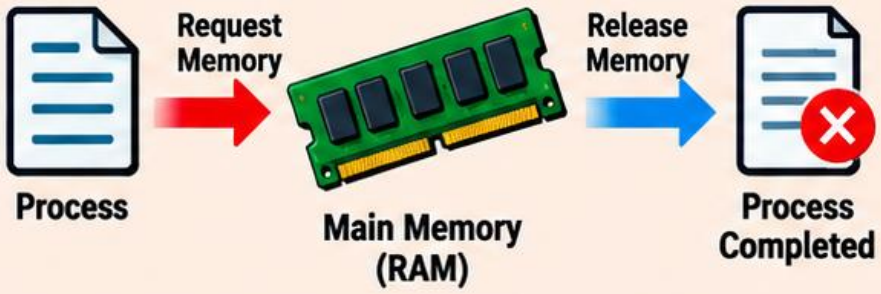
3. **Operating System Organization**
   In a multiprogramming computer, the operating system resides in a part of memory, and the rest is used by multiple processes.

4. **Subdivision of Memory**
   The task of subdividing the memory among different processes is called **Memory Management.**

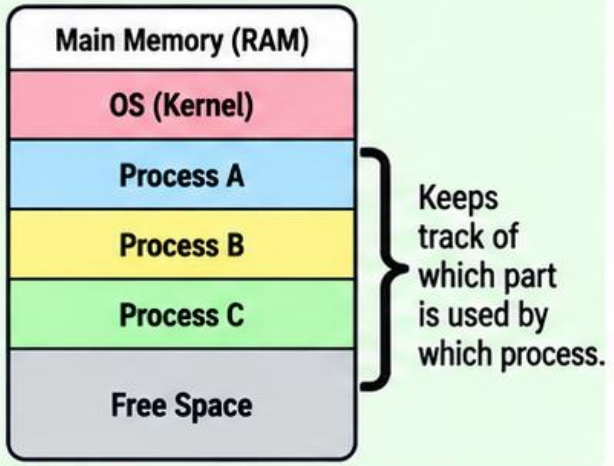
5. **Efficient Utilization of Memory**
   The main aim of memory management is to achieve efficient utilization of memory.

4.1.1 Why Memory Management is Required?

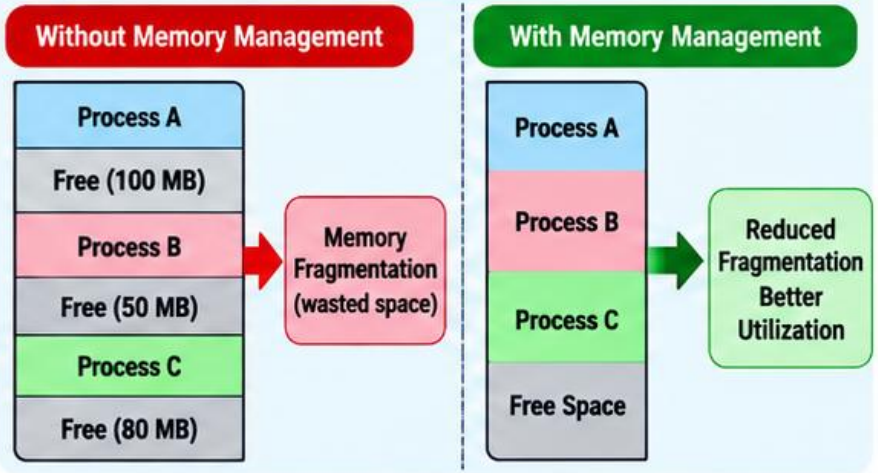
1 Allocate and de-allocate memory before and after process execution



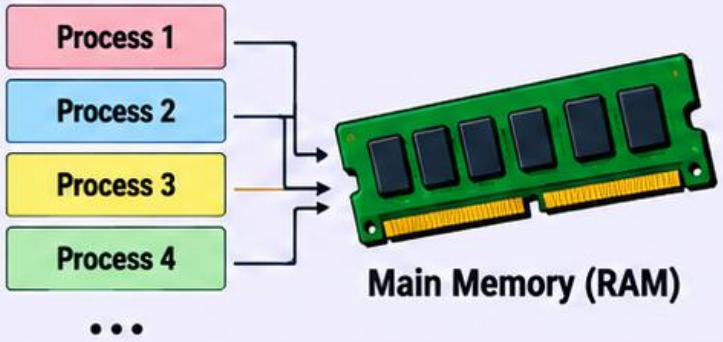
2 To keep track of used memory space by processes



3 To minimize fragmentation issues

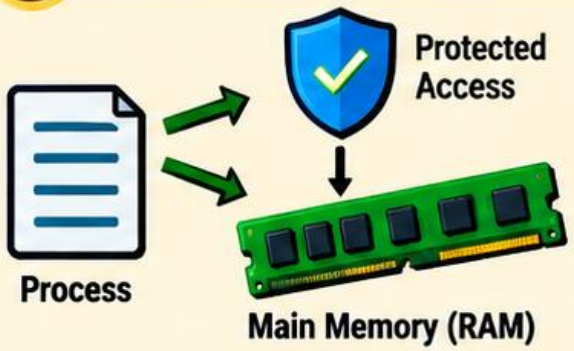


4 To proper utilization of main memory



- More processes in memory
- Efficient use of available space
- Better system performance

5 To maintain data integrity while executing of process



- Prevents one process from accessing another process's data.
- Ensures correct and consistent data.
- Avoids accidental or intentional modification of data.
- Improves system stability and security.

4.1.1 Logical vs. Physical Address Space

1 Logical Address



- Logical address is **generated by the CPU**.
- It is also referred to as **virtual address**.
- The set of all logical addresses used by a program composes the **logical address space**.

2 Physical Address



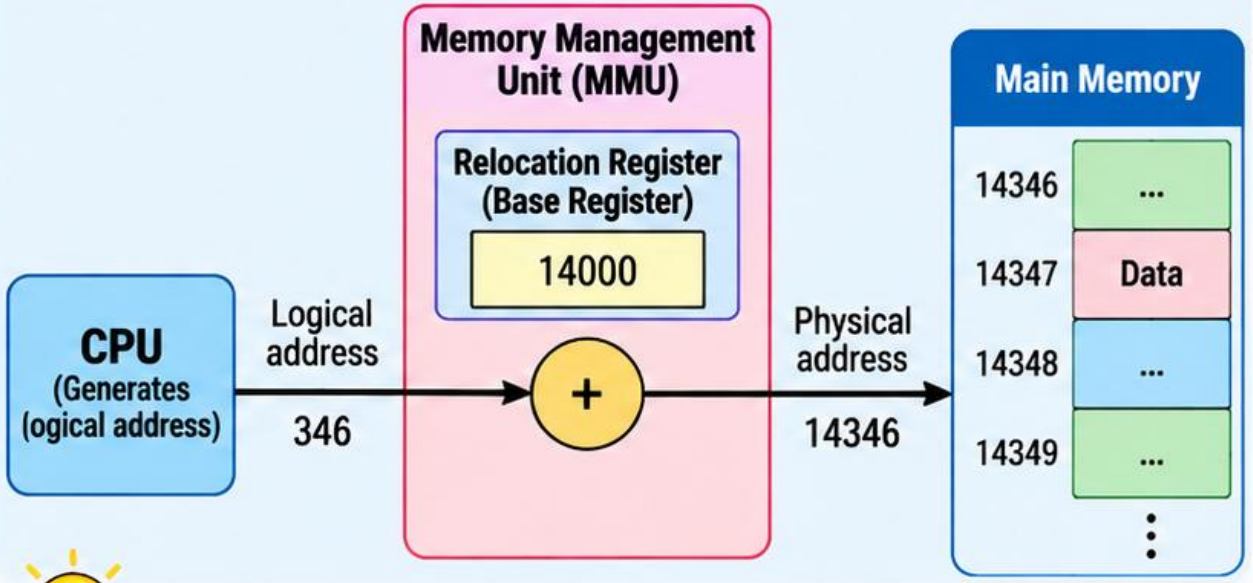
- Physical address is the address **seen by the memory unit**.
- The set of all corresponding physical addresses composes the **physical address space**.

3 Role of MMU and Relocation Register



- The run time mapping of logical to physical addresses is handled by the **memory-management unit (MMU)**.
- The base register is also known as a **relocation register**, whose value is added to every memory request at the hardware level.
- The user program deals with logical addresses, it **never sees the real physical addresses**.

4 Dynamic Relocation using Relocation Register



Key Takeaways

- Logical address → generated by CPU (virtual address)
- Physical address → actual address in main memory
- Logical address space → set of all logical addresses of a program
- Physical address space → set of all corresponding physical addresses
- MMU with relocation register → maps logical to physical address at run time
- User program → deals with logical addresses only

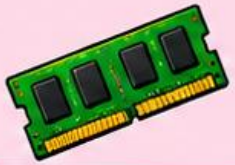
4.2 Memory management techniques





Memory Management Scheme (Techniques)

Contiguous Memory Management Scheme



In this scheme, each process is allocated a **single contiguous block** of memory.

Single Contiguous Memory Management Scheme

- Only one user process resides in memory along with the OS.
- Simple and easy to manage.



Single contiguous block

Multiple Partitioning

Memory is divided into **multiple partitions** to keep multiple **processes** in memory.

Fixed Partitioning

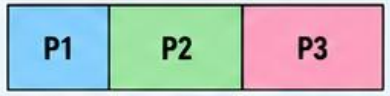
- Memory is divided into **fixed-size** partitions.
- Number and size of partitions are defined statically.



Fixed-size partitions

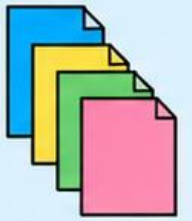
Dynamic Partitioning

- Partitions are created as per the size of the process.
- More flexible and better memory utilization.



Variable-size partitions

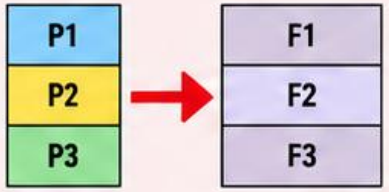
Non-Contiguous Memory Management Scheme



In this scheme, a process is **divided into multiple blocks** which can be stored at different locations in memory.

Paging

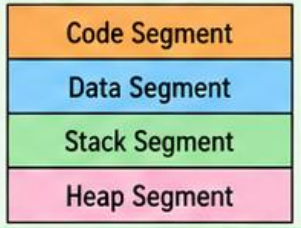
- Process is divided into **fixed-size pages.**
- Pages can be placed in any free frame of memory.
- Removes external fragmentation.



Pages Memory Frames

Segmentation

- Process is divided into **variable-size segments** (code, data, stack, etc.).
- Each segment has its own logical address space.
- Provides better support for modular programming.

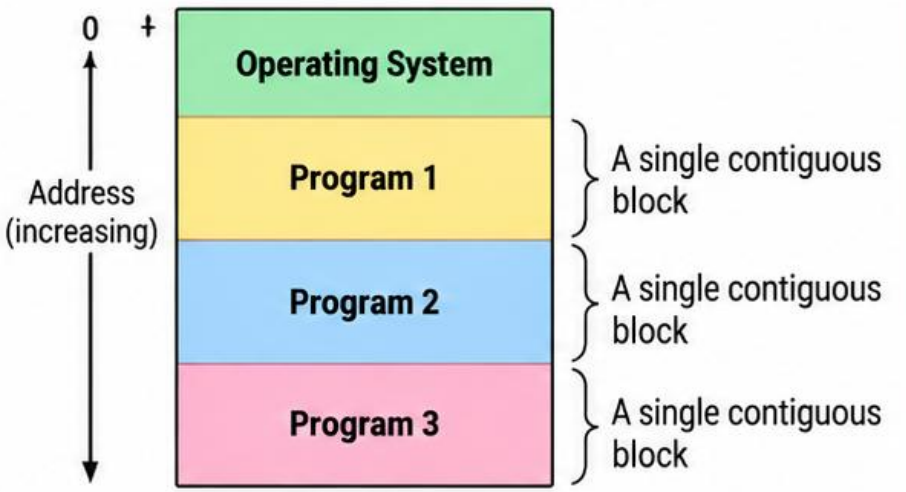


Contiguous memory management schemes

1 Contiguous Memory Management Scheme

- In a contiguous memory management scheme, each program occupies a **single contiguous block** of memory locations.
- A contiguous block means a set of memory locations with **consecutive addresses**.

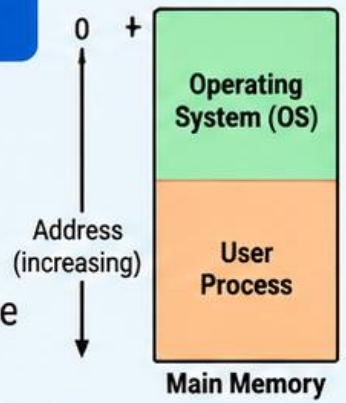
Memory Allocation (Contiguous)



2 Types of Contiguous Memory Management Schemes

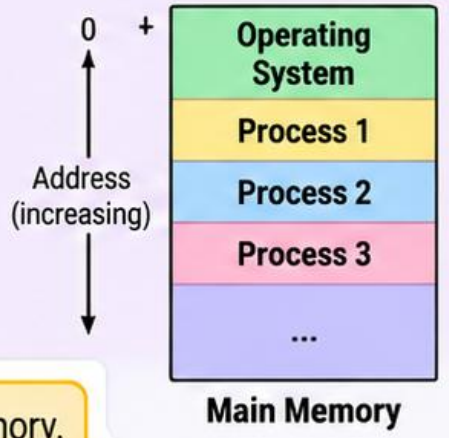
1 Single Contiguous Memory Management Scheme

- Main memory is divided into **two contiguous parts** or partitions.
- Operating system resides in one partition (usually lower memory) and the user process is loaded in the other partition.



2 Multiple Partitioning

- Main memory is divided into **multiple contiguous partitions** to keep multiple processes.
- Allows **more than one program** to be loaded and executed simultaneously (multiprogramming).



Key Takeaway: In contiguous memory management, each process is allocated a **single continuous block** of memory.

Multiple Partitioning in Memory Management

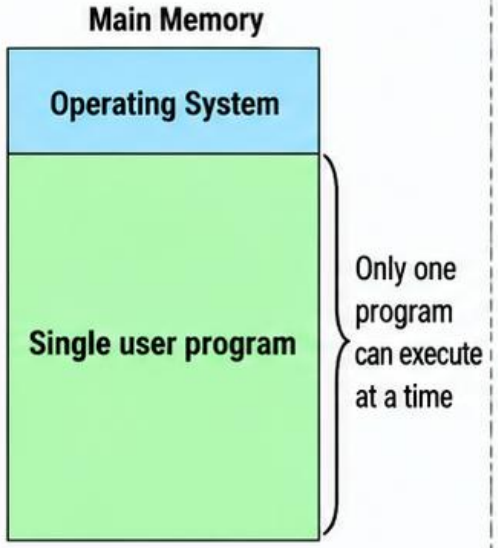


Why Multiple Partitioning ?

1. The single contiguous memory management scheme is inefficient as it limits computers to execute **only one program at a time**, resulting in **wastage in memory space and CPU time**.
2. The problem of inefficient CPU use can be overcome using **multiprogramming** that allows **more than one program** to run concurrently.
3. To switch between two processes, the operating system needs to **load both processes** into the main memory.
4. The operating system needs to **divide the available main memory into multiple parts** to load multiple processes into the main memory.
5. Thus multiple processes can reside in the main memory **simultaneously**.

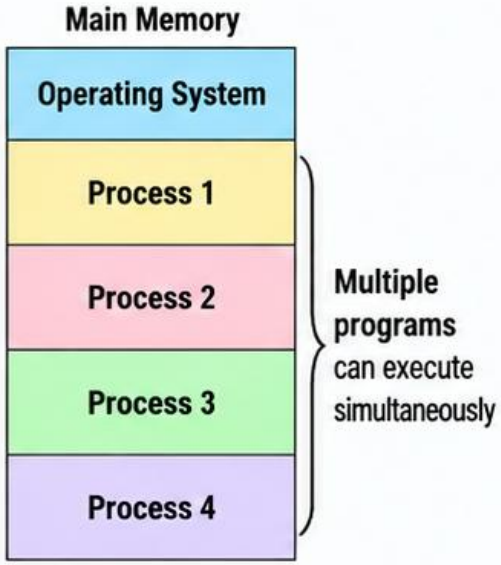
Single vs. Multiple Partitioning (Idea)

Single Partition (One program at a time)



Results in **wastage** of **memory and CPU time**.

Multiple Partitioning (More than one program)



Better CPU utilization and **memory usage**.

Multiple Partitioning Schemes & Types



The multiple partitioning schemes can be of two types:

1. Fixed Partitioning
2. Dynamic Partitioning

1 Fixed Partitioning
2 Dynamic Partitioning

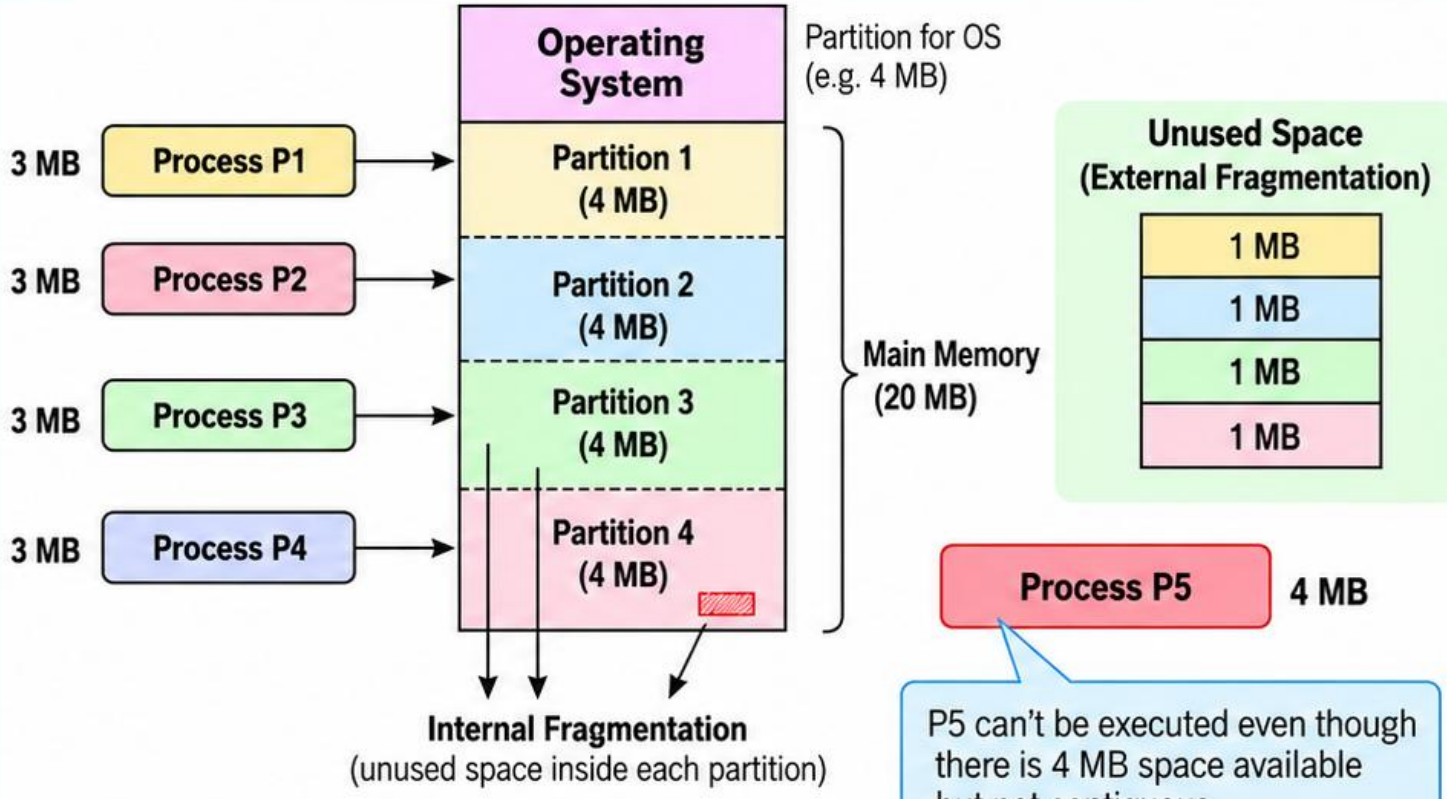
Fixed Partitioning

(A Multiple Partitioning Scheme)

Key Points

1. This is the **oldest and simplest technique** used to put more than one process in the main memory.
2. Memory is divided into several **fixed sized partitions**. Each partition may contain exactly one process.
3. The degree of multiprogramming is **bounded by the number of partitions**.
4. When a **partition is free**, a process is selected from the input queue and is loaded into the free partition.
5. When the **process terminates**, the partition becomes available for another process.
6. Number of partitions (non-overlapping) in RAM are **fixed** but size of each partition **may or may not be same**.
7. As it is contiguous allocation, **no spanning is allowed**. Partitions are made before execution or during system configuration.

Memory Layout in Fixed Partitioning (Example)



Fixed Partitioning

(Contiguous memory allocation)

Fixed Partitioning

(A Multiple Partitioning Scheme)



Advantages of Fixed Partitioning

1. Simple and **easy to implement**.
2. **Low overhead** (simple memory management).
3. **Fast** allocation and deallocation.
4. **No external fragmentation.**
5. Suitable for **small or batch systems.**

Disadvantages of Fixed Partitioning

1. **Internal fragmentation** (unused space inside each partition).
2. Degree of multiprogramming is **limited** (by number of partitions).
3. Partition size is **fixed** (may not be same).
4. If a process is larger than the partition size, it **cannot be allocated**.
5. Memory utilization may be **poor**.

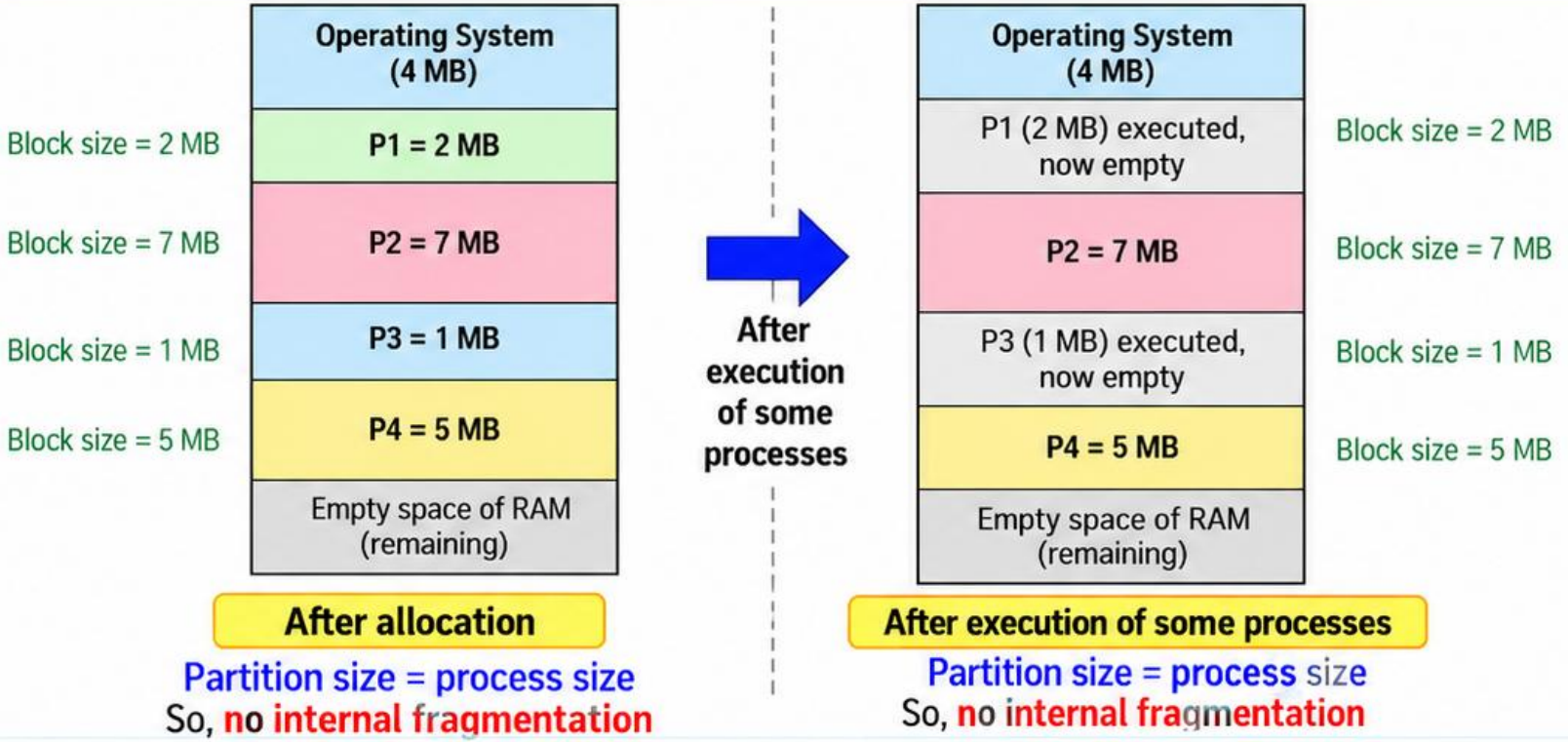
Dynamic Partitioning

(A Multiple Partitioning Scheme)

Concept of Dynamic Partitioning

1. Designed to **overcome the problems** of fixed partitioning scheme.
2. Each process occupies only **as much memory as it requires** when loaded for processing.
3. Initially, all memory is available for user processes and is considered **one large block or hole.**
4. Memory consists of a set of **holes of various sizes** scattered throughout the memory.
5. When a process arrives, the system searches for a **hole that is large enough** for this process.
6. If the hole is too large, it is **split into two parts** (one part allocated, remaining part returned to holes).
7. When a process terminates, its block is **released back** to the set of holes.
8. If **adjacent holes** exist, they are merged to form one larger hole.

Memory Layout in Dynamic Partitioning (Example)



After allocation

Partition size = process size
So, no internal fragmentation

After execution of some processes

Partition size = process size
So, no internal fragmentation

Advantages of Dynamic Partitioning

- Overcomes the limitations of fixed partitioning.
- Better memory utilization (allocates exact required size).
- No internal fragmentation.
- More flexible (partitions are created at run time).
- Suitable for varying process sizes.

Disadvantages of Dynamic Partitioning

- External fragmentation may occur (free memory is scattered in small holes).
- Uses complex algorithms to find suitable hole.
- Merging of adjacent holes is required.
- More overhead compared to fixed partitioning.

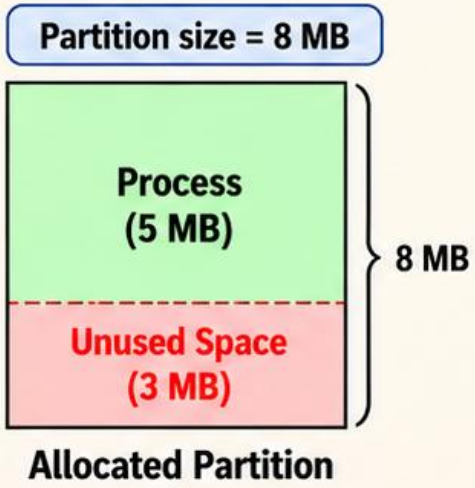
Prepared By-Prof. Vilas Rathod.

Internal Fragmentation vs External Fragmentation

Internal Fragmentation

- Internal fragmentation occurs when a **partition (fixed size)** is **larger than the process size**, and the remaining space inside the partition is **unused**.
- It happens in **fixed partitioning** (or when allocating a block larger than required).

Example :



3 MB is wasted inside the partition
(Internal Fragmentation)

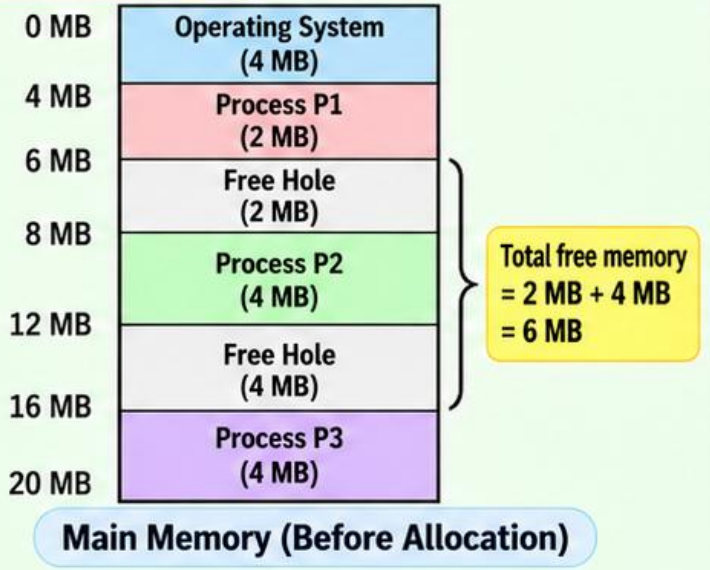
Key Points:

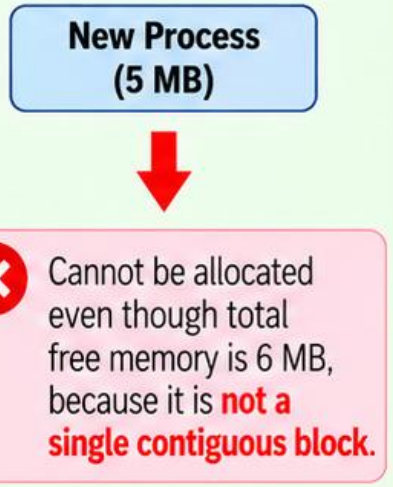
- Unused space is **inside** the allocated partition.
- Occurs due to **fixed partition size**.
- Increases memory wastage.
- Common in fixed partitioning.

External Fragmentation

- External fragmentation occurs when **total free memory is sufficient**, but it is divided into **multiple small, non-contiguous holes**, so a process requiring a **large contiguous block** cannot be allocated.
- It happens in **dynamic partitioning** (variable size partitions).

Example :





This is called **External Fragmentation**
(Free memory is in small non-contiguous holes)

Memory Allocation in Dynamic Partitioning



What is Memory Allocation in Dynamic Partitioning?

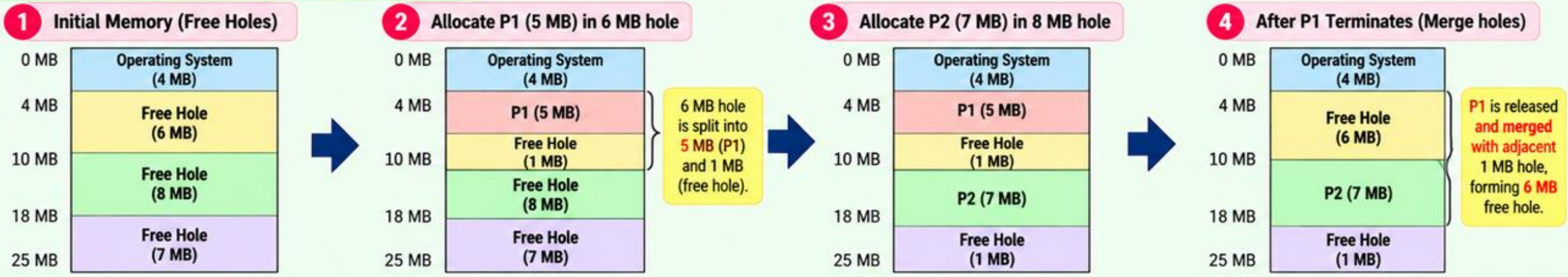
- Memory allocation in dynamic partitioning means assigning a **variable sized block (hole)** from the available free memory to a process **as per its requirement** at the time of execution.
- The operating system **searches the list of free holes** and selects a hole that is large enough to accommodate the process.
- If the selected hole is larger than the process size, it is **split into two parts**: one part is allocated to the process and the remaining part is kept as a new free hole.
- When the process terminates, its memory block is **released** and added back to the list of free holes. If adjacent free holes exist, they are **merged** to form a larger hole.

Memory Allocation in Dynamic Partitioning

Steps in Memory Allocation (Dynamic Partitioning)

1. **Process arrives** with memory requirement (e.g., 5 MB).
2. OS **searches the free holes** for a suitable hole (size ≥ 5 MB).
3. Selects one hole using an **allocation strategy** (First Fit / Best Fit / Worst Fit).
4. If hole is larger, **split the hole** and allocate required size and keep remaining part as a free hole.
5. When the process terminates, **release the block** and **merge** adjacent free holes if possible.

Example of Memory Allocation



Memory Allocation (in case of Dynamic Partitioning)

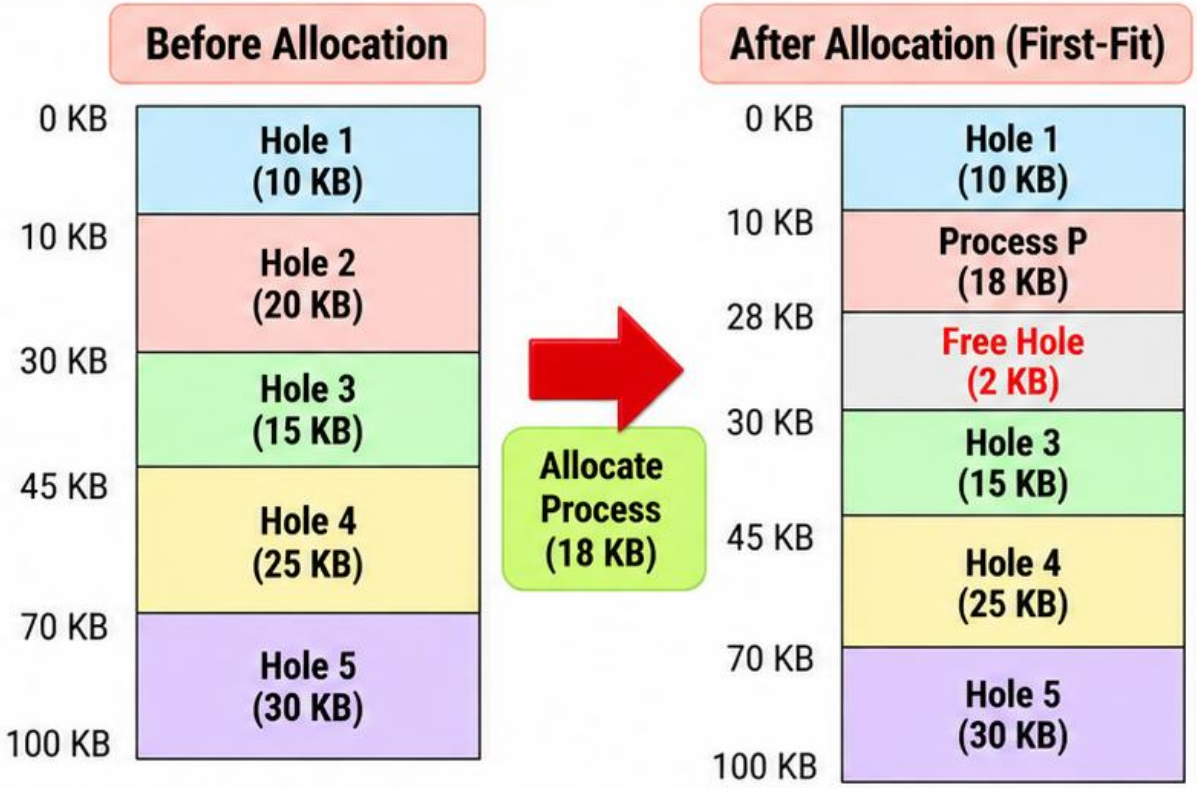
1. First-Fit (Memory Allocation)

- Allocate the **first hole** that is big enough.
- The first-fit algorithm searches for the **first free partition** that is **large enough** to accommodate the process.
- The operating system starts searching **from the beginning** and allocates the **first free partition** that is large enough to fit the process.
- **Example:** Suppose we have the following memory partitions (holes):

10 KB	20 KB	15 KB	25 KB	30 KB

Now, a process requests **18 KB** of memory. The operating system starts searching from the beginning and finds the **first free partition** of **20 KB**. It allocates the process to that partition and keeps the **remaining 2 KB** as free memory.

Example: First-Fit Allocation (Process size = 18 KB)



Result:

- First free partition **(20 KB)** is found.
- **18 KB** is allocated to the process.
- Remaining **2 KB** is kept as a free hole.

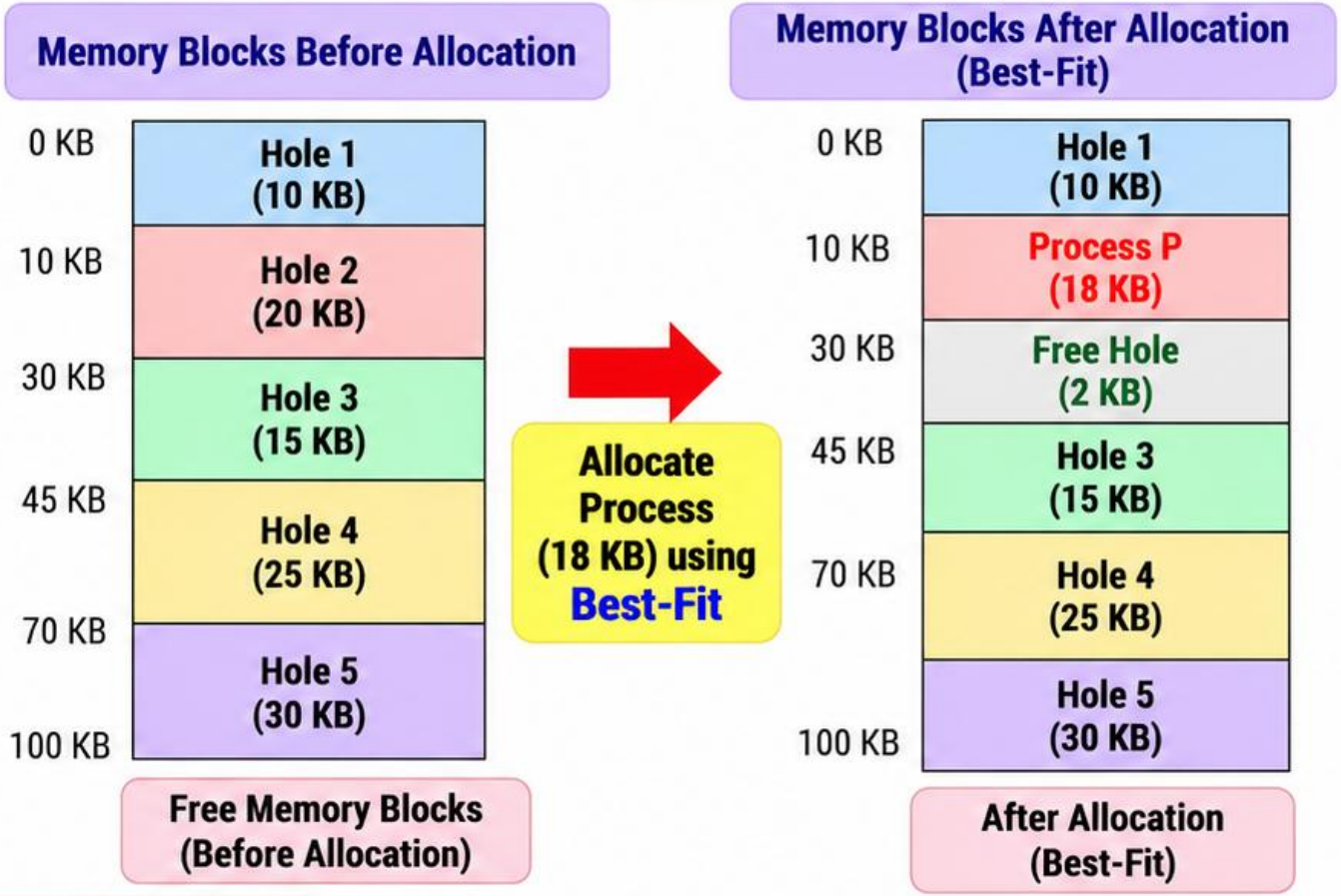
Memory Allocation in Dynamic Partitioning

2. Best-Fit Allocation

- The **best-fit algorithm** searches for the **smallest free partition** that is large enough to accommodate the process.
- The operating system searches the **entire memory** and selects the free partition that is **closest in size** to the process.
- It reduces the wasted space compared to first-fit but may create many small holes in memory.
- For example, suppose we have the following memory partitions (holes):

10 KB	20 KB	15 KB	25 KB	30 KB

A process requests **18 KB** of memory. The operating system searches for the **smallest free partition** that is larger than 18 KB. It finds the partition of **20 KB**.
It allocates the process to that partition and keeps the **remaining 2 KB** as free memory.



Result :

- Smallest free partition that is large enough for 18 KB is **20 KB**.
- **18 KB** is allocated to the process.
- Remaining **2 KB** is kept as a free hole.

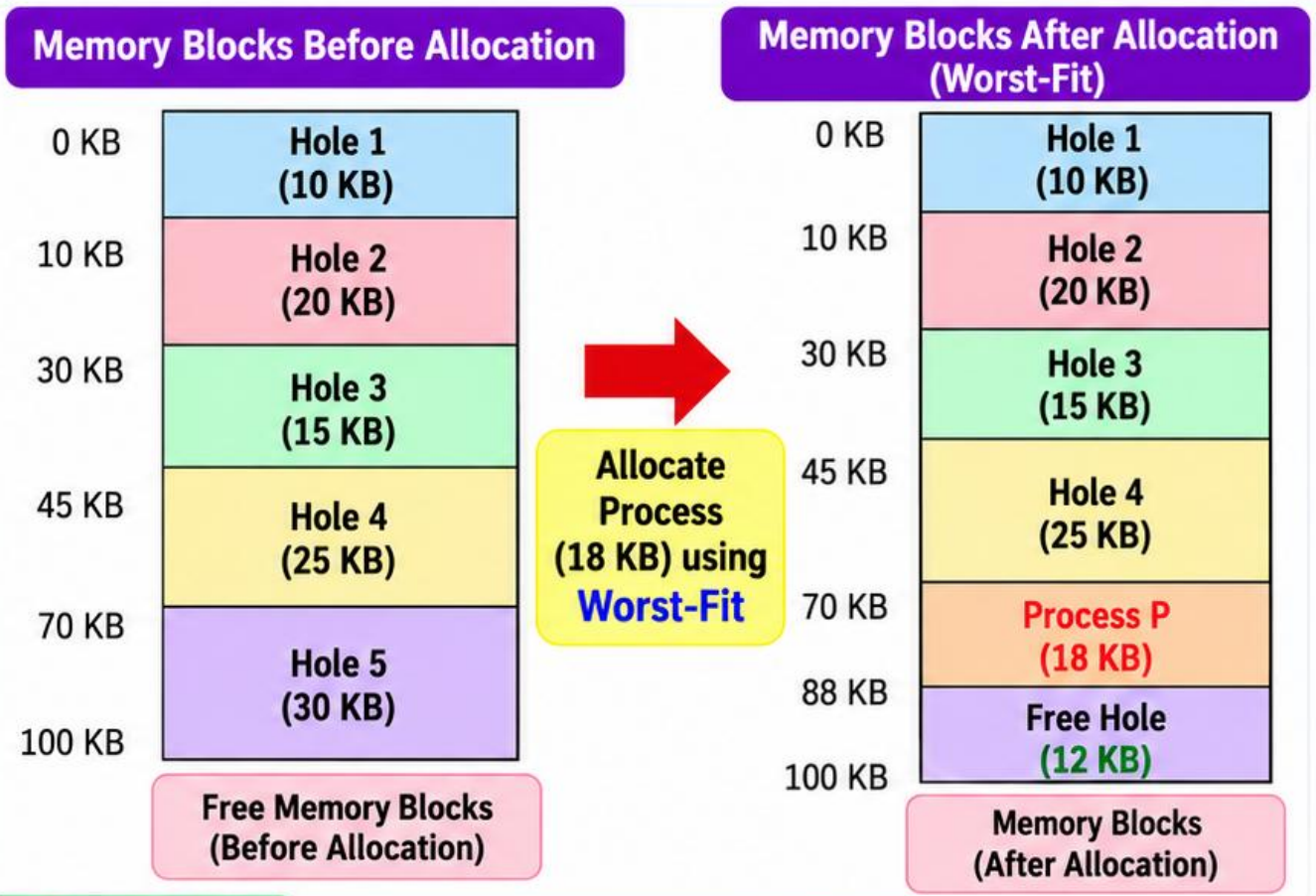
Memory Allocation (in case of Dynamic Partitioning)

3. Worst-Fit Allocation

- The **worst-fit** algorithm searches for the **largest free partition** and **allocates the process to it**.
- The operating system scans the entire memory and selects the free partition that is **largest in size**.
- It leaves a **large remaining free space**, which may be useful for future large processes.
- For example, suppose we have the following memory partitions (holes):

10 KB	20 KB	15 KB	25 KB	30 KB

Now, a process requests **18 KB** of memory.
The operating system searches for the **largest free** partition, which is **30 KB**.
It allocates the process to that partition and keeps the **remaining 12 KB** as free memory.



Result :

- Largest free partition is **30 KB**.
- **18 KB** is allocated to the process.
- Remaining **12 KB** is kept as a free hole.

Q. 1. A system has free memory blocks of **10 KB, 20 KB, 15 KB, 25 KB, and 30 KB**. A process requires **18 KB** of memory. Apply the **Best-Fit algorithm** and determine:
i. Which memory block will be selected?
ii. How much memory will remain free after allocation?

Q2. A memory contains free blocks of **10 KB, 20 KB, 15 KB, 25 KB and 30 KB**. A process of **18 KB** arrives. Apply **First-Fit, Best-Fit and Worst-Fit** separately and show the block selected and remaining space for each method.

Q. For the given memory blocks of 250 KB, 600 KB, 150 KB, 400 KB, and 350 KB, and processes of 140 KB, 320 KB, 275 KB, and 500 KB, apply the First Fit memory allocation technique and show the allocation result.

Memory Blocks (in KB)				
B1 250	B2 600	B3 150	B4 400	B5 350

Processes (in KB)			
P1 140	P2 320	P3 275	P4 500

First Fit Memory Allocation (Step-by-Step)

Process (P_i)	Process Size (KB)	Scan of Blocks (First Fit)	Block Allocated	Block Size (KB)	Remaining Size (KB)
P1	140	B1 (250) is the first block large enough.	B1	250	250 – 140 = 110
P2	320	B1 (110) is not enough. B2 (600) is the first block large enough.	B2	600	600 – 320 = 280
P3	275	B1 (110) not enough, B2 (280) not enough. B3 (150) not enough. B4 (400) is the first block large enough.	B4	400	400 – 275 = 125
P4	500	B1 (110), B2 (280), B3 (150), B4 (125) are not enough. B5 (350) is also not enough.	**Not Allocated**	–	–

First Fit Allocation Result

Process	P1	P2	P3	P4
Allocated Block	B1 (110 KB left)	B2 (280 KB left)	B4 (125 KB left)	**Not Allocated**

Q. For the given memory blocks of 250 KB, 600 KB, 150 KB, 400 KB, and 350 KB, and processes of 140 KB, 320 KB, 275 KB, and 500 KB, apply the First Fit memory allocation technique and show the allocation result.

Memory Blocks (in KB)				
B1	B2	B3	B4	B5
250	600	150	400	350

Processes (in KB)			
P1	P2	P3	P4
140	320	275	500

Best Fit Memory Allocation (Step-by-Step)

Process (P_i)	Process Size (KB)	Suitable Block(s) (Best Fit)	Block Allocated	Block Size (KB)	Remaining Size (KB)
P1	140	Blocks that can hold 140 KB: B1(250), B2(600), B3(150), B4(400), B5(350) → Best fit is B3 (150) [smallest sufficient block]	B3	150	150 – 140 = 10
P2	320	Blocks that can hold 320 KB: B1(250) – no, B2(600), B4(400), B5(350) → Best fit is B5 (350) [smallest sufficient block]	B5	350	350 – 320 = 30
P3	275	Blocks that can hold 275 KB: B1(250) – no, B2(600), B4(400) → Best fit is B4 (400) [smallest sufficient block]	B4	400	400 – 275 = 125
P4	500	Blocks that can hold 500 KB: B1(250) – no, B2(600) → Best fit is B2 (600)	B2	600	600 – 500 = 100

Best Fit Allocation Result				
Process	P1	P2	P3	P4
Allocated Block	B3 (10 KB left)	B5 (30 KB left)	B4 (125 KB left)	B2 (100 KB left)

Q. The following diagram represents the free memory blocks available in a system:
[10 KB] → [20 KB] → [15 KB] → [25 KB] → [30 KB]
A process requiring **18 KB** of memory arrives for execution.
Using the above memory layout, apply the following memory allocation techniques separately:

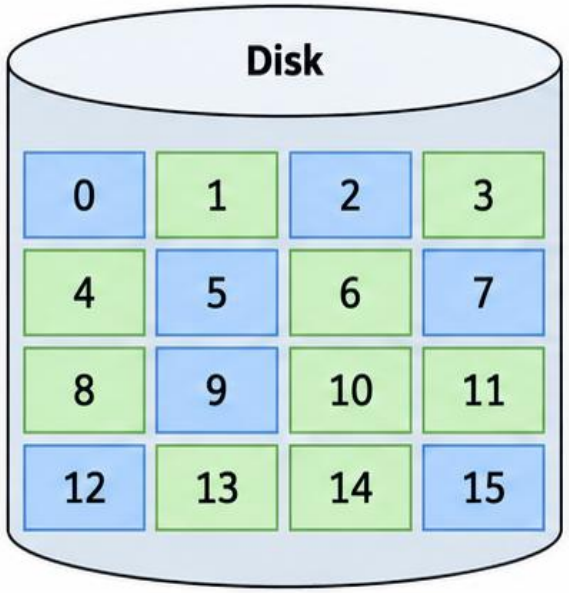
1. **First-Fit**
2. **Best-Fit**
3. **Worst-Fit**

For each technique, **identify the memory block selected and show the remaining free space after allocation using a suitable diagram.**

Free Space Management in Operating System

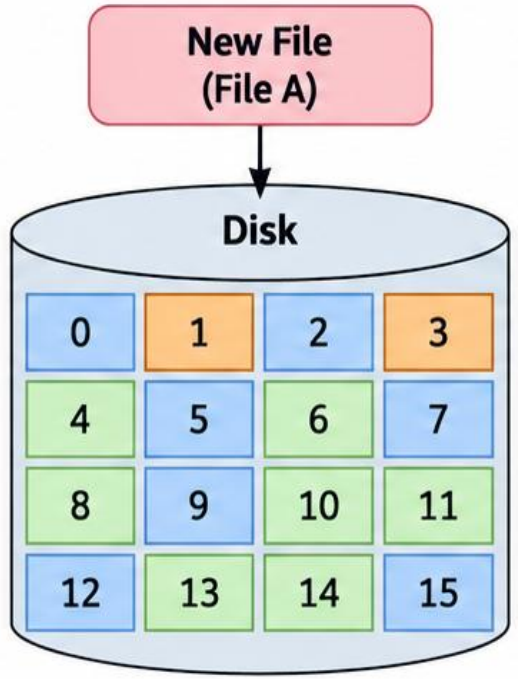
- It is used by an operating system to maintain information about **unallocated disk blocks** that are available for storing files and directories.
- A disk is divided into **fixed-size units called blocks**. When a **file is created**, the OS **allocates free blocks** to store its data. When a **file is deleted**, the OS **releases its blocks**, making them available for reuse.

1. Disk is divided into blocks



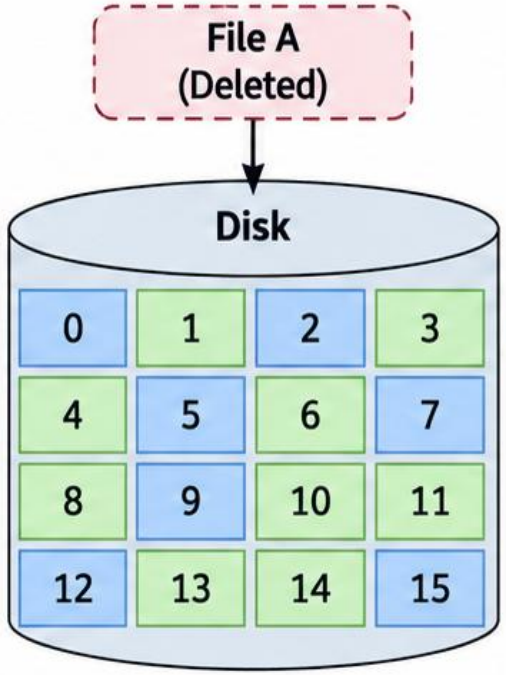
Disk divided into fixed-size blocks (e.g., 0 – 15)

2. File creation (Allocation)



OS allocates free blocks (e.g., 1 and 3) to store the new file.

3. File deletion (Release)



OS releases the blocks (1 and 3) making them available for reuse.

4. Free Space Management

OS maintains information about free blocks

List of Free Blocks (example)

1	3	4	6	8	10	11	13	14

- Allocated block (used by a file)
- Free block (unallocated, available for use)
- Newly allocated block (for a newly created file)

Free Space Management in Operating System

Why Is Free Space Management Required?

Free Space Management is required for the following reasons:

1. **Efficient disk allocation:** Helps the OS identify available blocks for new files.
2. **Better disk utilization:** Allows unused disk space to be reused.
3. **File creation:** Provides storage blocks when a new file is created.
4. **File extension:** Helps allocate additional blocks when an existing file grows.
5. **File deletion:** Returns released blocks to the pool of available storage.
6. **Improved performance:** Helps reduce the time needed to locate suitable free blocks.
7. **Prevention of allocation conflicts:** Helps ensure the same block isn't mistakenly allocated to multiple files.

Free Space Management Technique 1: Bit Vector (Bitmap)

1. Definition

A Bit Vector (Bitmap) is a free-space management technique in which each disk block is represented by a single bit. The bit indicates whether the corresponding block is free or allocated.

Convention (commonly used):
0 = Free block
1 = Allocated block

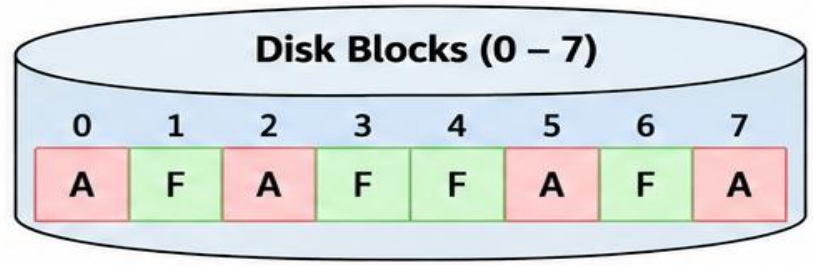
Note: The convention can be reversed in some implementations.

2. Working

1. The OS maintains a bitmap containing one bit for every disk block.
2. When a file needs storage, the OS searches the bitmap for free blocks.
3. Once a block is allocated, its bit is changed from 0 to 1.
4. When a file is deleted, the corresponding bit is changed from 1 to 0.
5. The updated bitmap represents the current allocation status.

3. Example – Initial State

Suppose a disk has 8 blocks.



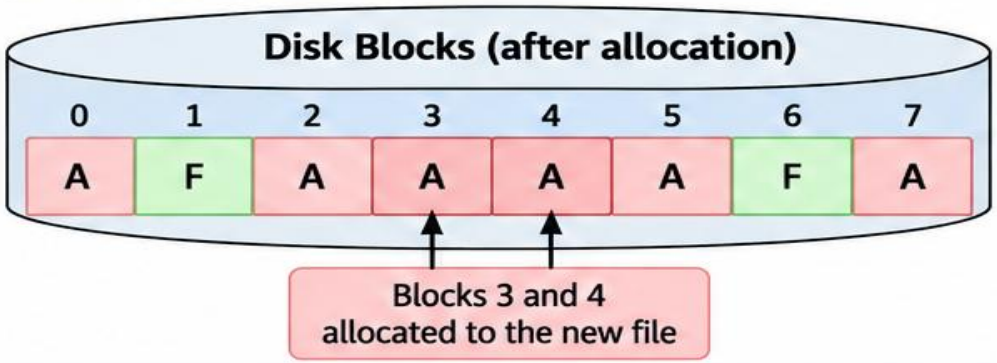
Block number	0	1	2	3	4	5	6	7
Status	A	F	A	F	F	A	F	A
Bitmap	1	0	1	0	0	1	0	1

A = Allocated F = Free

The bitmap is: **1 0 1 0 0 1 0 1**

4. Allocate Two Blocks (e.g., blocks 3 and 4)

A file requires two blocks. The OS finds free blocks (3 and 4) and allocates them.



5. Updated Bitmap

Block number	0	1	2	3	4	5	6	7
Status	A	F	A	A	A	A	F	A
Bitmap	1	0	1	1	1	1	0	0

The updated bitmap is:

1 0 1 1 1 1 0 1

6. Color Legend

Allocated block (1 in bitmap)

Free block (0 in bitmap)

Advantages & Disadvantages of using Bitmap

Advantages

- The main advantage of this approach is its **relative simplicity** and its **efficiency** in finding the first free block or n consecutive free blocks on the disk.
- **Easy to implement**.
- Requires **very little extra space** (only 1 bit per disk block).
- **Fast to find free blocks**, especially with bitwise operations.

Example

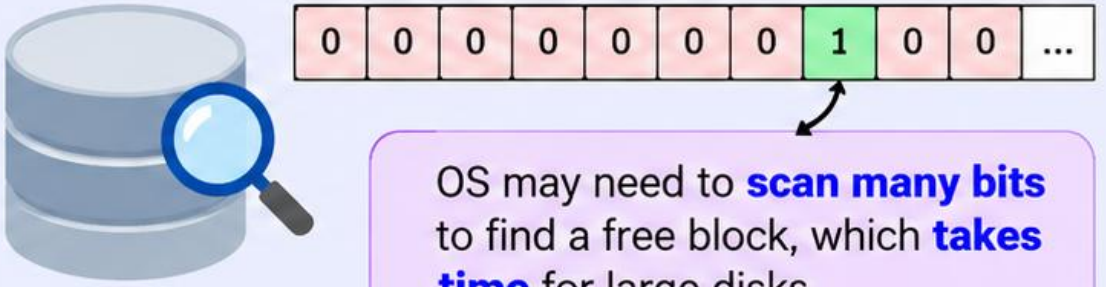
	Block0	Block1	Block2	Block3	Block4	Block5	Block6	Block7
Bitmap	1	0	1	1	0	0	1	0

1 = Free block **0** = Allocated block

Disadvantages

- **Decreases** the **degree of multiprogramming** as well as **throughput**.
- Takes a **huge amount of time for searching** which makes the system **inefficient** to some extent.

Why it can be inefficient?



Linked List for Dynamic Partitioning

Concept

- The better and the most popular approach to keep track the **free or filled** partitions is using **Linked List**.
- In this approach, all the **free disk blocks** are linked together, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- Each free block contains a **pointer to the next free block**, and so on.

Working

- The first free block is pointed to by a **free-space list head**.
- Each free block stores the address/number of the **next free block**.
- When a block is allocated, it is **removed from the linked list**.
- When a block is freed, it is **added back to the linked list**.

Example

- Consider a disk where blocks **2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26 and 27** are free and the rest of the blocks are allocated.
- We keep a **pointer to block 2** as the **first free block**.
- Block 2 would contain a pointer to block 3, which would point to block 4, and so on.

Linked Free-Space List on Disk

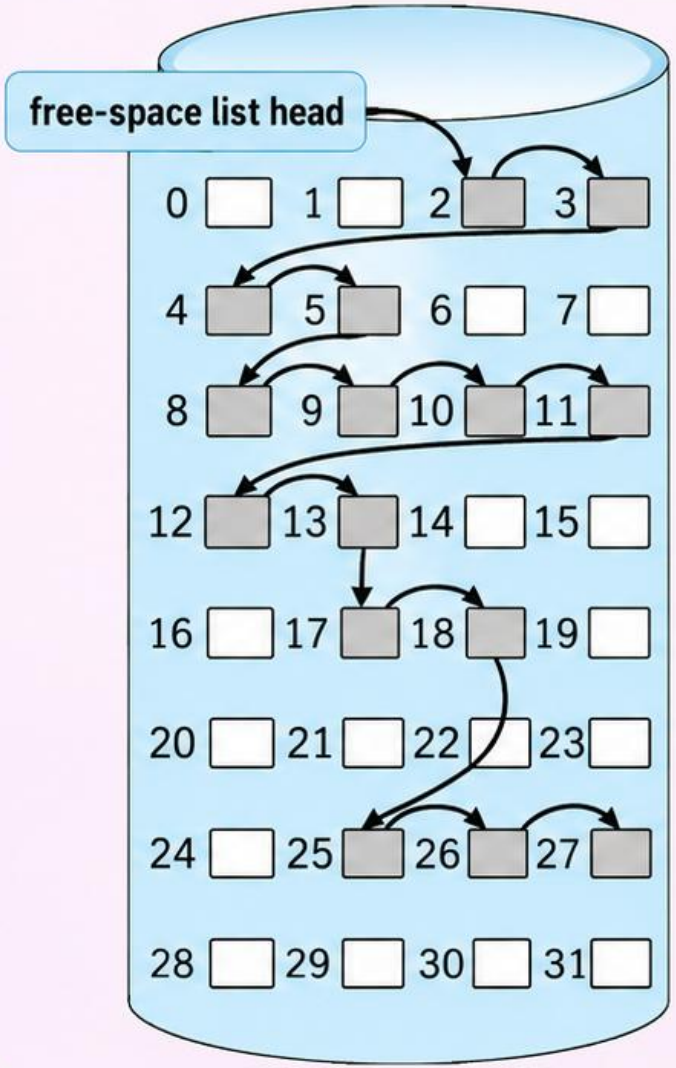


Fig. Linked free-space list on disk.

Legend

= Free space block

= Allocated block

Advantages & Disadvantages of Using Linked List

Advantages of Using Linked List

1. It is a **simple and efficient** approach to keep track of the **free or filled partitions**.



Simple & Easy to implement

2. Easy to allocate and free partitions (just **update pointers**).



Quick allocation and deallocation

3. Requires **very little extra memory** (only pointer information is stored).



Low memory overhead

4. Suitable for systems with **variable-size partitions** and frequent allocation/deallocation.



Flexible & dynamic

Disadvantages of Using Linked List

1. **Decreases** the degree of **multiprogramming** as well as system **throughput**.



Lower Throughput

2. Searching for a free block requires traversal of the list, which may take a **large amount of time,** especially in large disks.



Time-consuming search

3. **Access time is slower** compared to bit map, especially when the list is long.



Slower access time

Memory Management Techniques

Memory Management Scheme

Contiguous Memory Management Scheme
(Process is stored in one continuous block of memory)

Single Contiguous Memory Management Scheme

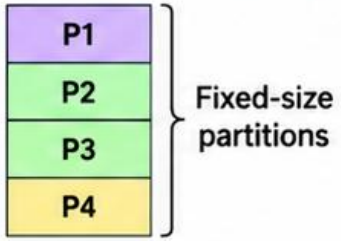
One process in one block



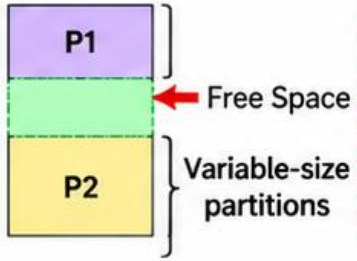
Contiguous Allocation

Multiple Partitioning

Fixed Partitioning

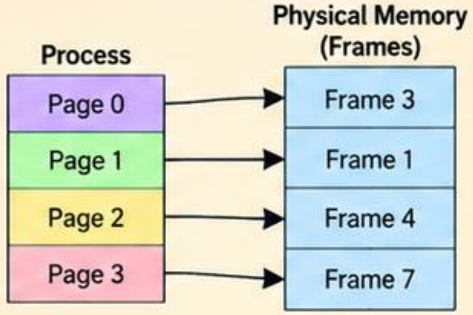


Dynamic Partitioning



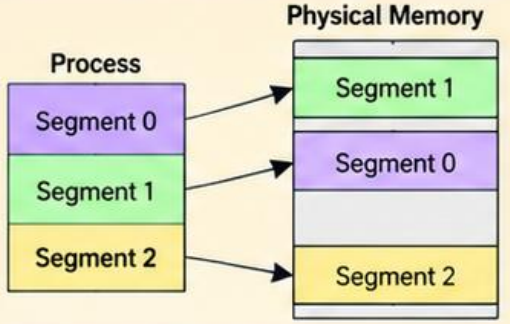
Non-Contiguous Memory Management Scheme
(Process is stored in multiple blocks at different memory locations)

Paging



Process is divided into fixed-size pages and stored in different frames (non-contiguous).

Segmentation



Process is divided into variable-size segments and stored in different memory locations (non-contiguous).

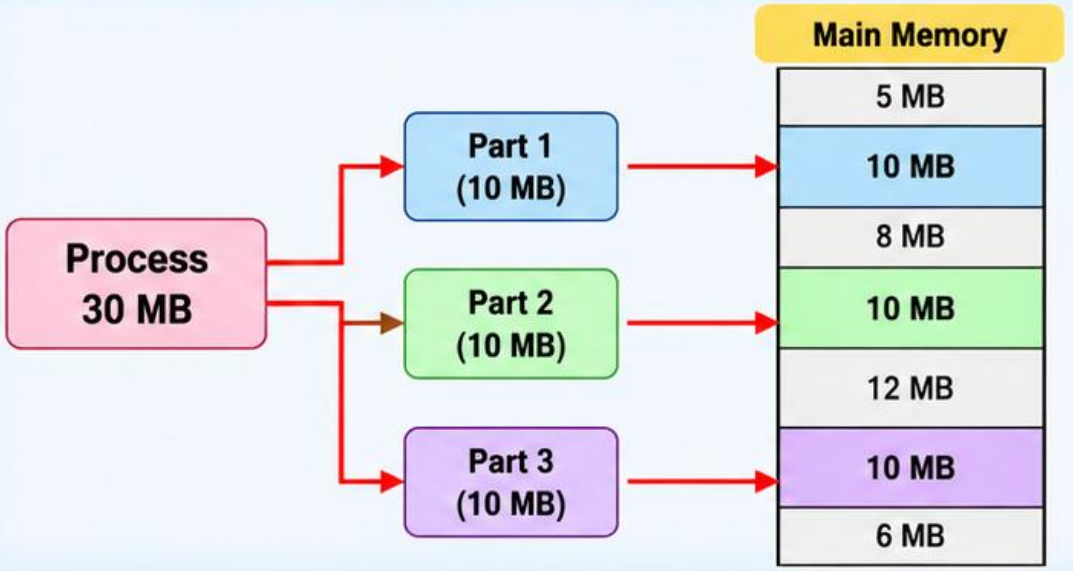
Non-Contiguous Types (Highlighted)

Non-Contiguous Memory Management Schemes

Concept

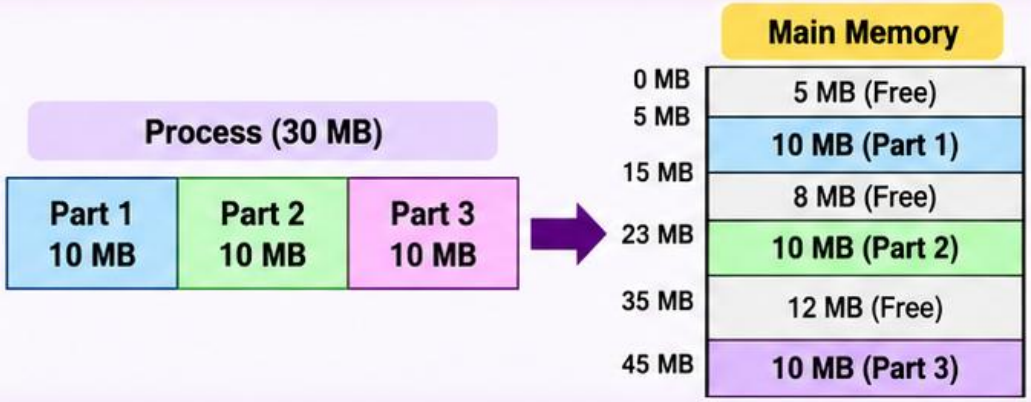
- In a Non-Contiguous memory management scheme, the **program is divided into different blocks** and **loaded at different portions of the memory** that need not necessarily be adjacent to one another.
- Non-contiguous memory allocation also **reduces memory waste** caused by **internal and external fragmentation** because it utilizes the memory gaps left by internal and external fragmentation.

Non-Contiguous Allocation in Memory



Example: Executing a 30 MB Process Using Non-Contiguous Allocation

- Suppose you have a 30 MB process, but you don't have a single 30 MB contiguous space available for execution.
- Using a non-contiguous memory management scheme, the 30 MB process is divided into **3 parts (10 MB each)** and loaded into different memory locations.



- Process is divided into different blocks.
- Blocks are loaded at different memory locations.
- Blocks need not be contiguous.

Paging: Concept, Structure of Page Table

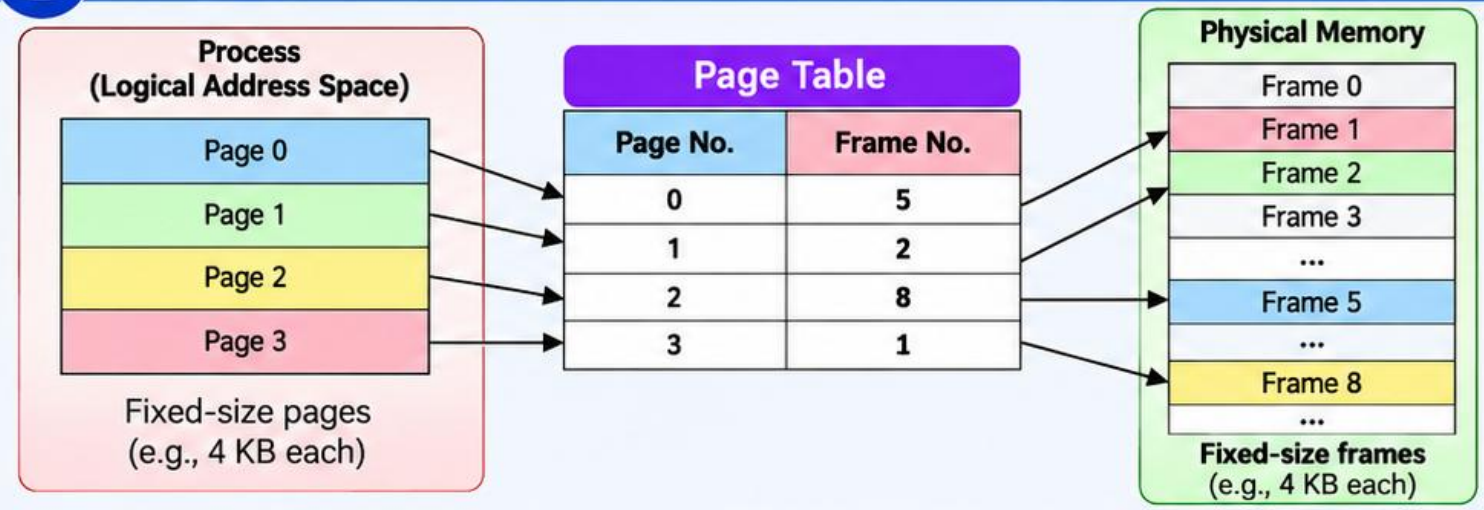
Concept of Paging

- Paging is a memory management scheme that eliminates the need for a **contiguous allocation** of physical memory.
- Paging is a **storage mechanism** used in OS to retrieve processes from **secondary storage** to **main memory** as **pages**.
- The basic purpose of paging is to separate each process into **pages** and divide physical memory into **fixed-size blocks** called **frames**.
- This scheme permits the physical address space of a process to be **non-contiguous**.

Structure of Page Table

- The physical memory is divided into fixed-size blocks called **frames**, which are the **same size** as the pages used by the process.
- The process's logical address space is divided into fixed-size blocks called **pages**, which are the **same size** as the page frames.
- When a process requests memory, the operating system **allocates** one or more page frames to the process and **maps** the process's logical pages to the physical page frames.

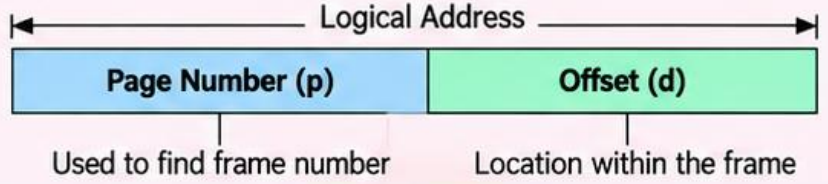
Paging Mechanism



Address Translation using Page Table

A logical address is divided into two parts:

- **Page number (p)** – used to index the page table
- **Offset (d)** – displacement within the page



Physical Address = (Frame Number × Page Size) + Offset

Important Terms in Paging

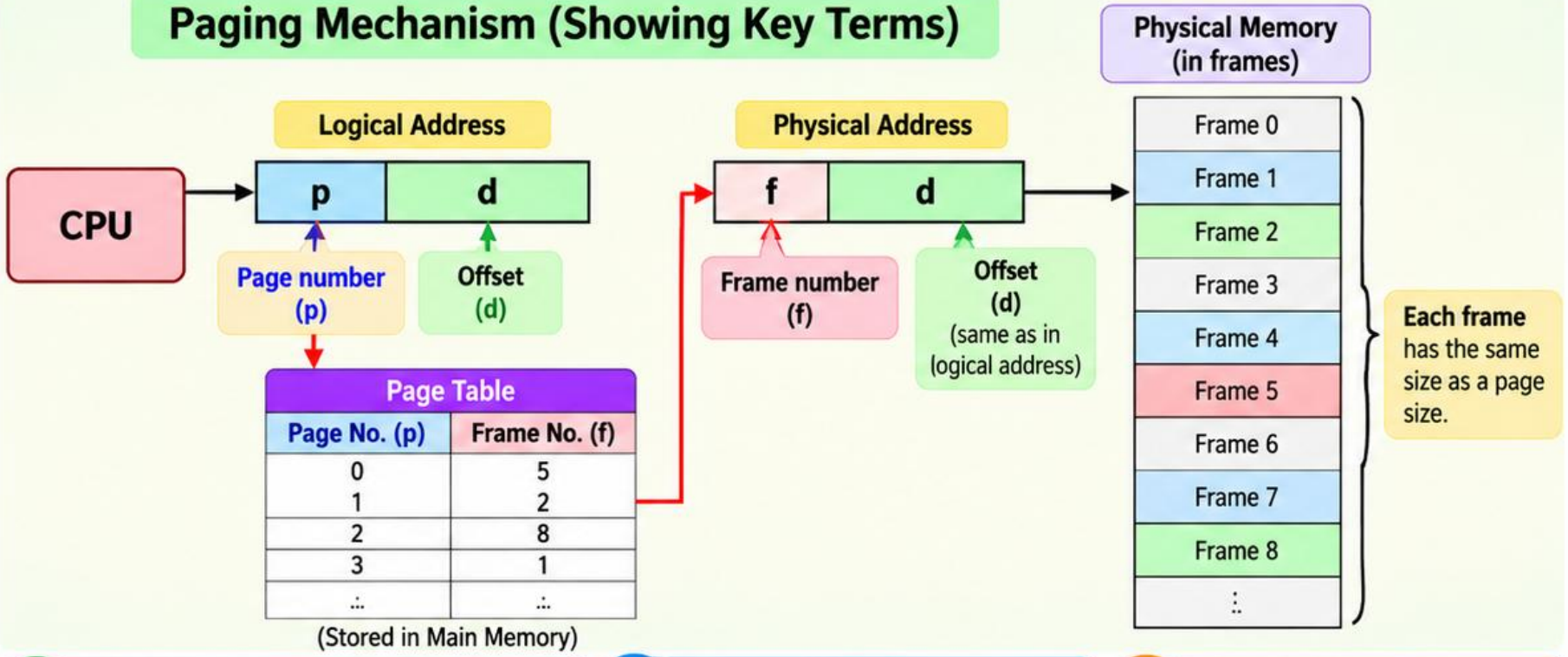
1 Page Number

- A page number (p) is the **index of a page** in the logical address space of a process.
- The logical address is divided into two parts: page number (p) and offset (d).
- It is used to **look up the page table** to find the corresponding frame number.

2 Page Table

- A page table is a data structure used by the Operating System to **map each page number to a frame number** in physical memory.
- Each entry in the page table contains the frame number and other control information (valid bit, protection bits, etc.).

Paging Mechanism (Showing Key Terms)



3 Frame Number

- A frame number (f) is the index of a frame in the physical memory.
- It is obtained from the page table using the page number.
- It indicates where the page is stored in physical memory.

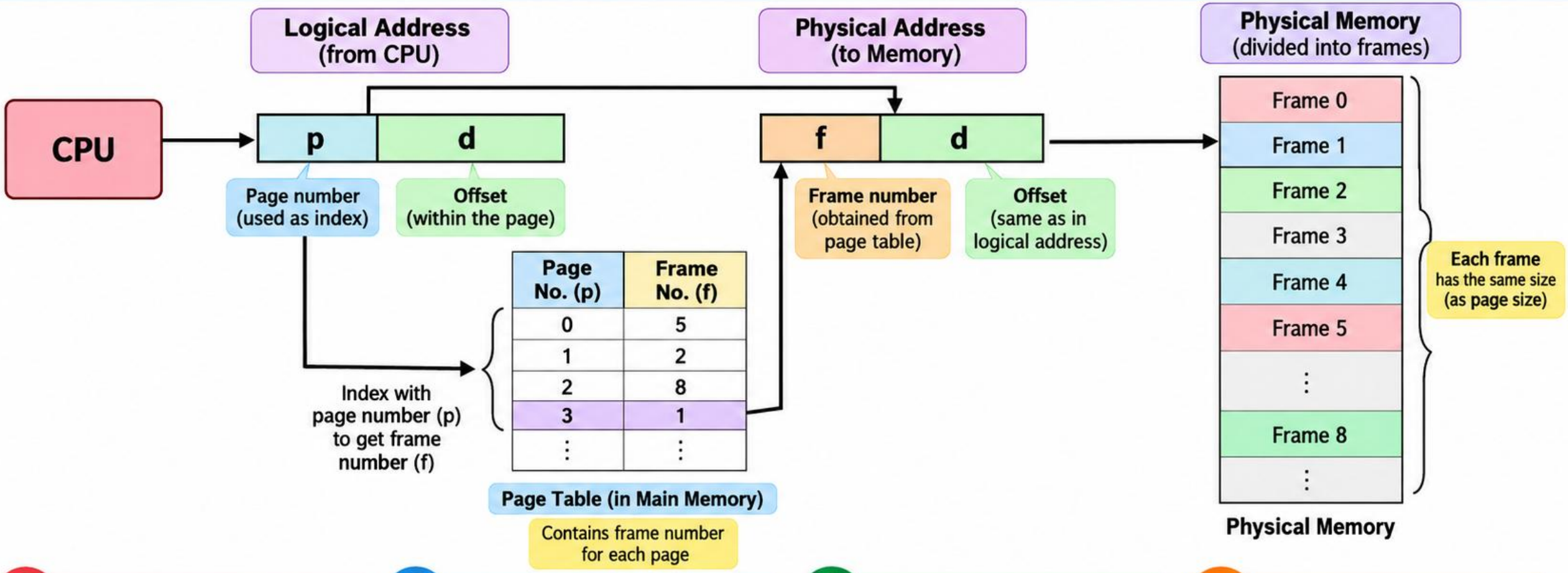
4 Page Fault

- A page fault occurs when the **required page is not present in main memory**.
- The operating system then brings the page from secondary storage (disk) to main memory and updates the page table.
- Until the page is loaded, the process is blocked (cannot continue execution).

5 Offset

- An **offset** (d) specifies the **exact location (byte/word) within a page**.
- The **offset** is the same in both logical and physical addresses.
- If the page size is 1 K words (2^{10}), then the offset requires 10 bits.

Paging Hardware: Diagram



1 CPU generates a logical address

Logical address is divided into:

- Page number (p)
- Offset (d)

2 Use page number to index page table

Page number (p) is used to look up the page table, which returns the corresponding frame number (f).

3 Form physical address

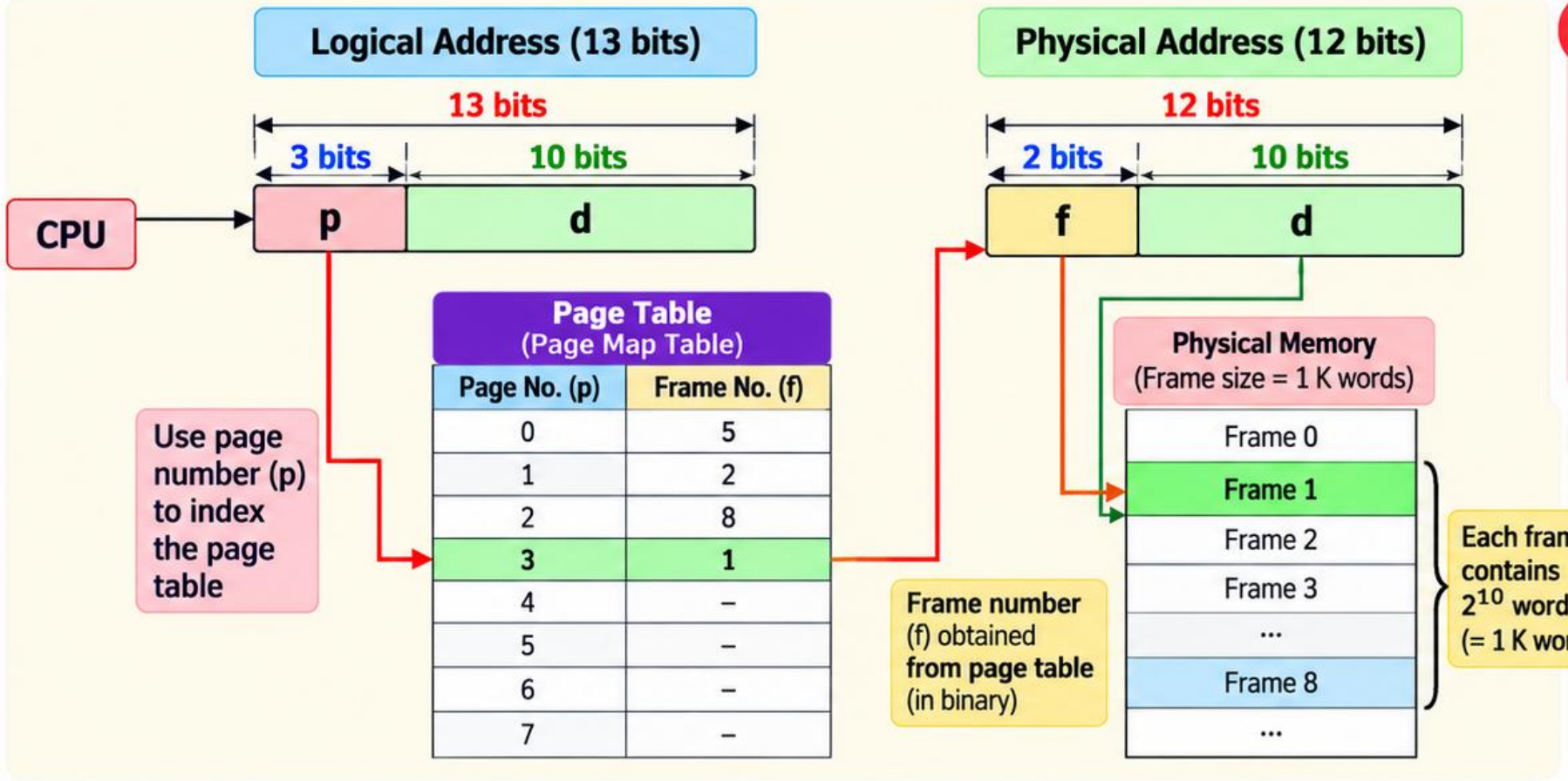
Replace page number (p) with frame number (f). Offset (d) remains the same.

Physical Address = (f , d)

4 Access main memory

The physical address (f , d) points to the required byte in the physical memory.

Paging Concept with Example



Example

Let us consider an example:

- Physical Address = 12 bits, then Physical Address Space = 4 K words
- Logical Address = 13 bits, then Logical Address Space = 8 K words
- Page size = Frame size = 1 K words (assumption)

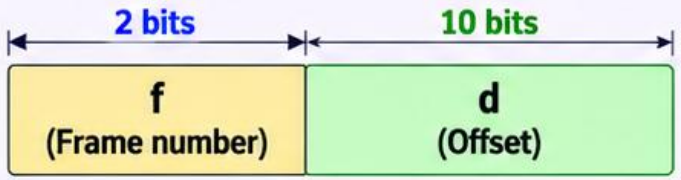
Calculations

- **Number of frames** = Physical Address Space / Frame size = 4 K / 1 K = **$4 = 2^2$**
- **Number of pages** = Logical Address Space / Page size = 8 K / 1 K = **$8 = 2^3$**

Logical Address Format (13 bits)



Physical Address Format (12 bits)

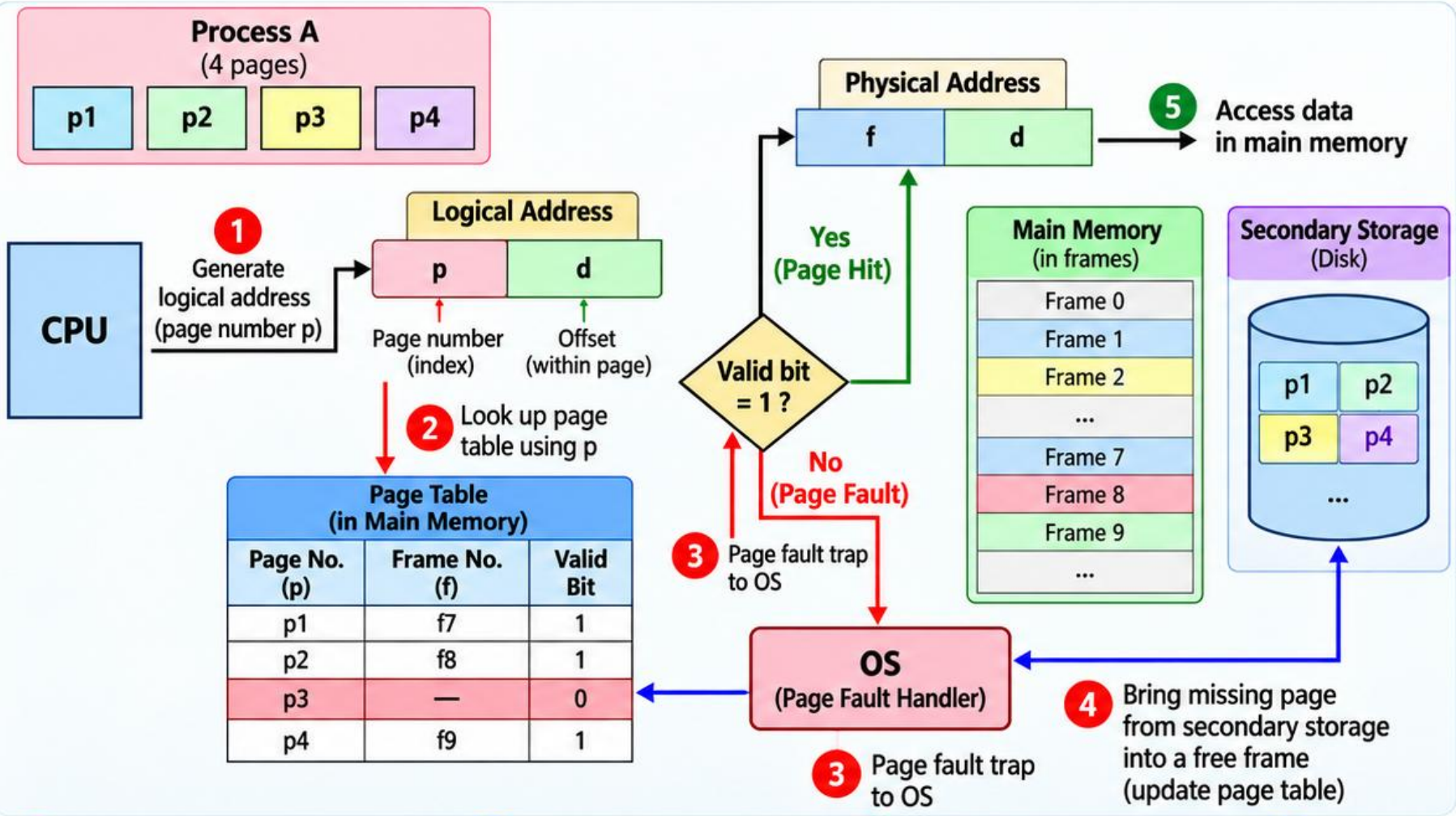


Address Translation

Physical Address = (Frame Number × Page Size) + Offset

- **Frame Number (f)** = value obtained from page table using **page number (p)**
- **Offset (d)** = displacement within the page

Page Faults & Page Fault Handling (Thrashing)



What is a Page Fault?

- A page fault is a type of error that occurs when a program tries to access data that is **not currently in the main memory (RAM)**.
- A page fault occurs when a program attempts to access data or code that is in its address space, but is not currently located in the system RAM.
- Access to a page marked invalid (**invalid bit = 0**) causes a page fault.

Why Page Fault Occurs?

- The required page is not present in main memory.
- **The valid bit** in the page table is **0 (invalid)**.
- The paging hardware detects this and generates a trap to the operating system.

1 Generate Logical Address

CPU generates a logical address consisting of a **page number (p)** and an **offset (d)**.

2 Check Page Table

- Use page number (p) to look up the page table and check the valid bit.

3 Page Fault Trap

- If **valid bit = 0**, paging hardware generates a **page fault trap** to the operating system.

4 Handle Page Fault

- OS brings the missing page from secondary storage (disk) into a free frame and updates the page table.

5 Access Memory

- After updating the page table, the instruction is restarted and data is accessed from the main memory.

THANK YOU